DeepShop: A Benchmark for Deep Research Shopping Agents Supplementary Material

Yougang Lyu[1], Xiaoyu Zhang[2], Lingyong Yan[3],
Maarten de Rijke[1], Zhaochun Ren[4], and Xiuying Chen[5]

[1] University of Amsterdam, The Netherlands [2] University College London, United Kingdom
[3] Baidu Inc., China [4] Leiden University, The Netherlands
[5] Mohamed bin Zayed University of Artificial Intelligence, United Arab Emirates
youganglyu@gmail.com

Appendix

A Details of Task Complexity Distribution Analysis

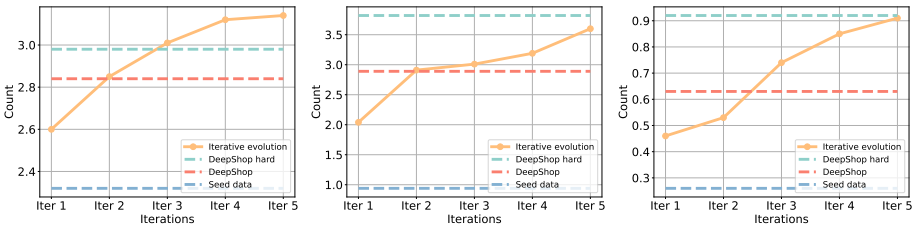
To further validate that complexity evolution introduces consistent task difficulty across different domains in the DeepShop benchmark, we conduct a detailed analysis of overall task complexity distribution across domains.

For the overall task complexity distribution, we calculate the mean and standard deviation of the total number of product attributes, search filters, and sorting preferences. As shown in the Table 1, DeepShop exhibits a higher average number of constraints and lower standard deviations across domains compared to the seed datasets. These results suggest that DeepShop presents greater overall task difficulty while maintaining improved cross-domain consistency.

Table 1. Overall task complexity distribution across domains.

Domain	Electronic	Fashion	Sports	Books	Home	Avg.	Std
Seed data	3.43	3.60	3.29	4.25	3.50	3.61	0.33
DeepShop	6.22	6.30	6.32	6.26	6.11	6.24	0.07

Analysis of query complexity evolution. The complexity evolution strategy progressively enhances query complexity by incorporating additional product attributes, search filters, or sorting preferences. We perform a fine-grained analysis of query evolution across these three dimensions. Regarding product attributes, as depicted in Figure 1a, the average number of product attributes per query exhibits a steady increase throughout the iterations. Ultimately, the average number of product attributes in DeepShop surpasses the seed data by 0.52, while the hard subset contains an additional 0.66 attributes on average. In terms of search filters, Figure 1b illustrates a consistent increase in the average number of filters per query across iterations. At the final iteration, DeepShop queries include, on



(a) Product attribute evolution. (b) Search filter evolution. (c) Sorting preference evolution.

Fig. 1. Analysis of query complexity evolution.

average, 1.95 more filters than the seed queries, with the hard subset further increasing this difference to 2.88 filters on average. Similarly, the evolution of sorting preferences, illustrated in Figure 1c, shows an upward trajectory. The final average sorting preferences per query exceed the seed data by 0.37, and this increment is further pronounced in the hard subset, where queries contain an additional 0.66 sorting preferences on average.

B Details of Evaluation

B.1 GPT-4o for Evaluation

This section details the GPT-4o prompt used for fine-grained evaluation. As shown in Figure 2, we adapt the prompt from prior work [3] to assess whether the web agent satisfies the requirements related to product attributes, search filters, and sorting preferences. For each trajectory, GPT-4o receives the subqueries, the agent's action history, and a maximum of 15 screenshots, and is prompted to determine whether the task was successfully completed.

B.2 Human evaluation

To validate the reliability of GPT-4o-based evaluation, we conduct a human evaluation on 50 randomly sampled trajectories from WebVoyager. Following previous work [3,6], we hire human annotators with at least an undergraduate degree, and each sampled trajectory is independently evaluated by a minimum of two annotators. In cases of disagreement, a third annotator is consulted, and the final judgment is determined by majority vote. The evaluation is conducted based on the full execution trace of the agent, including the task description, operation history, and screenshots, as shown in Figure 3. Each human annotator labels the success of each subgoal—product attributes, search filters, and sorting preferences—as well as the overall task completion. The average Cohen's kappa among human annotators is 0.82, indicating substantial inter-annotator agreement. We then compute the agreement rate between human and GPT-4o assessments.

Additionally, since deep research systems do not expose intermediate screenshots or action histories, human annotators are allowed to access the Amazon website to verify whether the returned links satisfy the specified requirements. Due to limited use of the

deep research systems, we evaluated its performance on 30 randomly sampled queries. To ensure consistency, we evaluate the accuracy based on the first recommended item when multiple products are returned.

B.3 Significance test between web agents

We conducted paired t-tests to compare the overall task success rates between **Browser Use** and four baseline models: **Simple RAG**, **Agent-E**, **SeeAct**, and **WebVoyager**. The results reveal that **Browser Use significantly outperforms Simple RAG and web agent baselines** ($p < 0.05$), supporting the claim that direct interaction with the online web environment can substantially improve agent performance over static retrieval methods such as Simple RAG. Furthermore, the comparison highlights fundamental limitations in grounding capabilities across different agent types. HTML-based agents like Agent-E lack access to visual context, often missing essential cues such as product color or spatial layout. On the other hand, vision-based agents such as SeeAct and WebVoyager rely on set-of-mark prompting, which suffers from segmentation errors: interactive elements are frequently misclassified, and key sections—like customer review areas—remain unsegmented, making it difficult to apply filters like rating thresholds.

C Details of Baselines

- **Simple RAG method**: We implement a simple retrieval-augmented generation (RAG) baseline by combining GPT-4o with Google Search. The user query is first submitted to Google Search, and the top-ranked webpage is selected. GPT-4o (version 2024-08-06) then generates a final response conditioned on the screenshot of the retrieved page. We use the Serper API[1] to programmatically access Google search results.
- **Web agents**: For fair comparison, all web agents are instantiated using GPT-4o (version 2024-08-06) as the underlying language model. **Agent-E** [1] is an HTML-based agent that adopts a hierarchical planner-actor architecture, augmented with flexible DOM tree distillation and a denoising mechanism to improve decision accuracy.[2] **SeeAct** [7] exploits the multi-modal capabilities of large language models (LLMs), integrating visual perception with structured web-based interactions.[3] **WebVoyager** [3] also leverages multi-modal reasoning and introduces a set-of-mark prompting scheme that guides the agent to first generate intermediate thoughts before selecting final actions.[4] **Browser Use** [4] is an open-source web agent framework that combines visual understanding with HTML structure parsing to support robust web navigation and interaction.[5]
- **Deep research systems**: Since we cannot strictly constrain deep research systems to operate on specific shopping websites, we include explicit site constraints in the

[1] https://serper.dev/
[2] https://github.com/EmergenceAI/Agent-E
[3] https://github.com/OSU-NLP-Group/SeeAct
[4] https://github.com/MinorJerry/WebVoyager
[5] https://github.com/browser-use/browser-use

[System prompt]
As an evaluator, you will be presented with three primary components to assist you in your role:

1. Web Task Instruction: A clear and precise natural language directive that specifies an online shopping activity to be executed. The instruction may involve locating products that meet certain attribute requirements (e.g., color, size, brand), applying specific search filters (e.g., price range, customer ratings, availability), or fulfilling user-defined sorting preferences (e.g., lowest price, newest arrivals, best sellers). Tasks may also include verifying product details, comparing offers, or checking for shipping and return policies, depending on the scenario.

2. Result Screenshots: This is a visual representation of the screen showing the result or intermediate state of performing a web task. It serves as visual proof of the actions taken in response to the instruction.

3. Result Response: This is a textual response obtained after the execution of the web task. It serves as textual result in response to the instruction.

-- You DO NOT NEED to interact with web pages or perform actions such as conducting searches on websites.
-- You SHOULD NOT make assumptions based on information not presented in the screenshot when comparing it to the instructions.
-- Your primary responsibility is to conduct a thorough assessment of the web task instruction against the outcome depicted in the screenshot and in the response, evaluating whether the actions taken align with the given instructions.
-- NOTE that the instruction may involve more than one task, for example, locating the garage and summarizing the review. Failing to complete either task, such as not providing a summary, should be considered unsuccessful.
-- NOTE that the screenshot is authentic, but the response provided by LLM is generated at the end of web browsing, and there may be discrepancies between the text and the screenshots.
-- Note the difference: 1) Result response may contradict the screenshot, then the content of the screenshot prevails, 2) The content in the Result response is not mentioned on the screenshot, choose to believe the content.

You should elaborate on how you arrived at your final evaluation and then provide a definitive verdict on whether the task has been successfully accomplished, either as 'SUCCESS' or 'NOT SUCCESS'.

[User prompt]
TASK:
{subquery}

Result Response:
{answer}

15 screenshots at the end:
{screenshots}

Fig. 2. Prompts for GPT-4o evaluation

You will be presented with a web shopping task.
For each task, you will receive three subqueries, along with the web agent's action history and corresponding screenshots. Your goal is to evaluate the agent's performance across three specific dimensions: product attributes, search filters, and sorting preferences.
Please note: if a subquery is labeled as None, you do not need to assess that particular aspect.

Definitions of the three subqueries are as follows:
1. Product attributes, referring to concrete product characteristics users may specify to express detailed intent
2. Search filters, representing categorical or numerical constraints commonly used on e-commerce platforms
3. Sorting preferences, indicating desired result orderings, such as price or popularity.

Task:
{Query}

Product Attribute Requirement:
{Subquery1}

Search Filter Requirement:
{Subquery2}

Sorting Preference Requirement:
{Subquery3}

Agent Action History:
{Action}

Screenshots:
{Screenshots}

Fig. 3. Instructions for human evaluation.

prompt to guide the search process. The prompt format used for these systems is illustrated in Figure 4. **Gemini Deep Research** [2] is an AI assistant integrated into Google's Gemini Advanced platform. We evaluate the Gemini 2.0 Flash model with deep research capabilities.[6] **OpenAI Deep Research** [5] is an agentic system powered by OpenAI's reasoning models. We evaluate the o3 model with deep research enabled.[7]

Prompt:
Your task is to help find relevant products on Amazon website (https://www.amazon.com/?language=en_US&currency=USD) based on the following question: {Query}

Please strictly follow the rules:
1. Do not contact the user for further information. Provide answers directly without asking any questions or seeking clarification.
2. Include Amazon product links for all recommended items.

Fig. 4. Prompts for deep research systems.

D Details of Implementation

D.1 Prompts for query diversity and complexity evolution

Details for the prompts used in Diversity$(\cdot)$, and Complexity$(\cdot)$ are provided. Figures 5, 6, and 7 display the prompts for query diversity, complexity evolution and detailed complexity evolution strategy, respectively.

D.2 Implementation details

We evaluate open-source agents—Agent-E, SeeAct, WebVoyager, and Browser Use—within real-time web environments. Agent-E, SeeAct, and Browser Use are executed via Playwright, while WebVoyager leverages Selenium. To control computation cost and prevent excessive exploration, we limit each agent to a maximum of 15 steps per task. All agents are powered by `gpt-4o-2024-08-06` as the underlying language model. For automatic evaluation, we also adopt `gpt-4o-2024-08-06` as evaluators, with temperature set to 0 to reduce response variance and enhance reproducibility. The agents differ in their perception mechanisms: Agent-E and SeeAct utilize full-page screenshots, whereas WebVoyager and Browser Use operate on the visible viewport only.

[6] https://blog.google/products/gemini/google-gemini-deep-research/
[7] https://openai.com/index/introducing-deep-research/

I want you act as a Prompt Creator.
Your goal is to draw inspiration from the #Given Prompt# to create a brand new prompt.
This new prompt should be in the web shopping domain but tailored for different specific products in the {Randomly selected product category} amazon product field.
The LENGTH and complexity of the #Created Prompt# should be similar to that of the #Given Prompt#.
The #Created Prompt# must be reasonable and must be understood and responded by humans.
'#Given Prompt#', '#Created Prompt#', 'given prompt' and 'created prompt' are not allowed to appear in #Created Prompt#

#Given Prompt#:
{Original query}

#Created Prompt#:

Fig. 5. Prompts for query diversity evolution.

I want you act as a Prompt Rewriter for web shopping.
Your objective is to rewrite a given prompt into a more complex version to make those web shopping agents a bit harder to handle.
But the rewritten prompt must be reasonable and must be understood and responded by humans.
You should complicate the given prompt using the following method:
{Randomly selected evolution strategy}
You should try your best not to make the #Rewritten Prompt# become verbose, #Rewritten Prompt# can only add 10 to 20 words into #The Given Prompt#.
'#The Given Prompt#', '#Rewritten Prompt#', 'given prompt' and 'rewritten prompt' are not allowed to appear in #Rewritten Prompt#

#Given Prompt#:
{Original query}

#Created Prompt#:

Fig. 6. Prompts for query complexity evolution.

Product attribute evolution:
Enhance #The Given Prompt# by integrating detailed product attributes that detail user needs. Please specifies concrete values for one product attribute (e.g., brand, model, price range, color, size, weight, or unique features) based on your knowledge about this product, ensure that these exact details are incorporated into the query instead of using generic placeholder terms.

Search filter evolution:
Enhance #The Given Prompt# by integrating detailed product constraints that capture user needs. Please specifies concrete values for constraints—such as a minimum customer rating (e.g., above 4.0 or 4.5 stars), a minimum number of customer reviews (e.g., 100, 300, 500, or 1000), shipping options like free delivery, new arrival time frames (e.g., released in the last 30 or 90 days), return policies (e.g., free returns), or warranty information (e.g., includes a 1-year warranty) based on your knowledge about amazon website, ensure that these exact values are used in the query rather than generic terms.

Sorting preference evolution:
Enhance #The Given Prompt# by integrating a specific product filtering requirement for web shopping. Find the top product based on one of the following criteria: lowest price, highest user rating, newest arrival, or best seller ranking.

Fig. 7. Prompts for complexity evolution strategies.

E Details of Error Analysis

- **Web agents are limited by grounding ability.** As shown in Figure 8, agents struggle to accurately ground interface elements. For example, button 39 related to user rating was not properly segmented, preventing the agent from selecting a specific rating range. Buttons 31–37 and 41–44 were rendered too densely and overlapped, making interaction difficult. Additionally, the sorting button on the right was incorrectly split into two buttons 16 and 17, which may confuse the agent during execution.
- **Web agents often lack state assessment and replanning capabilities.** As illustrated in Figure 9, when the agent enters a product detail page to verify a 1-year warranty, it fails to reassess its state upon realizing that the requirement is unmet. Instead of returning to the search results page, the agent continues to scroll within the current page, inefficiently attempting to locate an alternative product.
- **Web agents are limited by action space.** As shown in Figure 10, the agent attempts to filter cameras within the $100–$300 price range but fails to do so due to its inability to interact with dynamic UI elements such as price sliders. Instead, it clicks the adjacent "Go" button without adjusting the slider values, resulting in ineffective filtering. This highlights a fundamental limitation of current agents: the constrained action space prevents them from performing fine-grained interactions required in realistic web environments.
- **Web agents lack the ability to learn from execution.** As illustrated in Figure 11, we present screenshots of the web agent across four different tasks. A consistent failure pattern emerges: the agent repeatedly misuses the retriever to query filtering or sorting constraints, despite these functionalities being accessible only via the designated filter or sorter UI components. Due to the retriever's inability to interpret such structural intents, it often returns irrelevant results contaminated by noise.

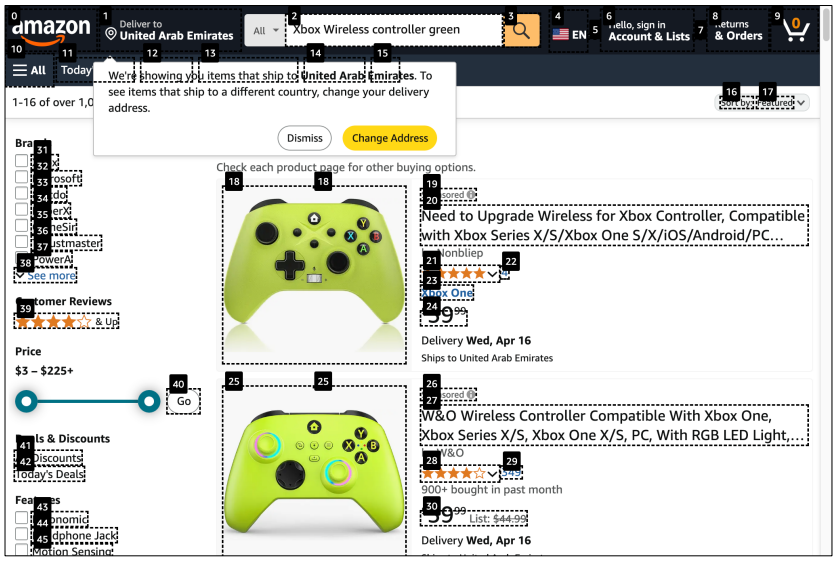


Fig. 8. Limited grounding ability of web agents.

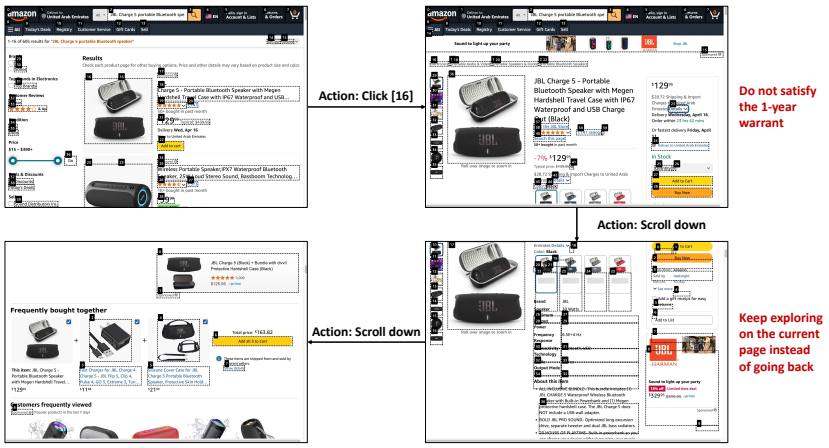


Fig. 9. Illustration of web agent's failure to reassess and replan.



Fig. 10. Illustration of the web agent's failure to apply the price filter during task execution.

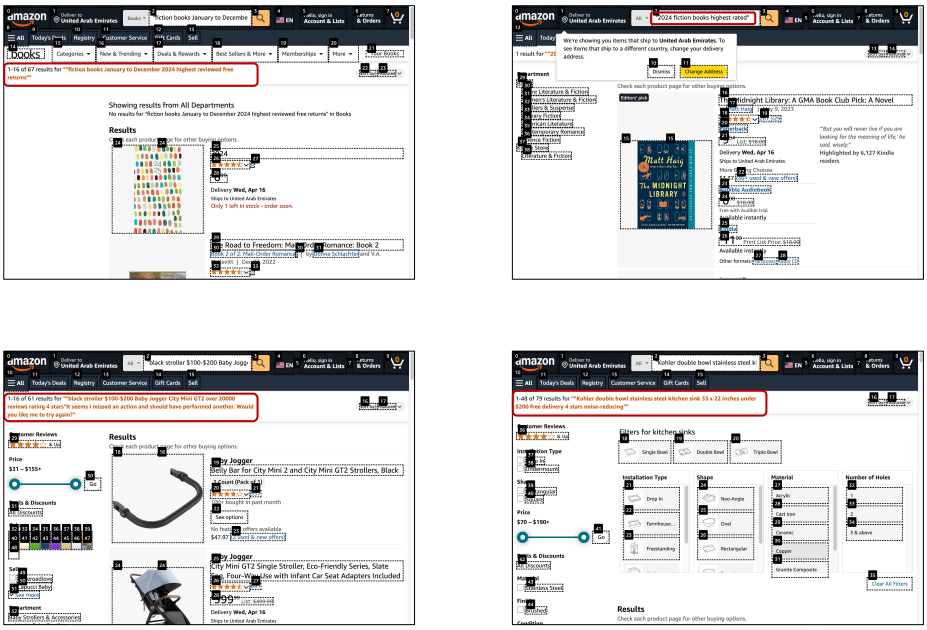


Fig. 11. Illustration of the web agent's failure to learn from execution.

This repetition across tasks highlights the agent's lack of execution-time learning, preventing it from abstracting past mistakes and adapting its behavior accordingly.

– **Deep research systems are limited by hallucination errors.** As shown in Figure 12, the task requires *identifying a Women's Vintage Floral Maxi Dress in Navy Blue, Size: Medium, and explaining the return policy if free returns are available*. However, as shown in Figure 13, the first link returned by the OpenAI deep research system points to a product only available in Large and XX-Large sizes. Despite this mismatch, the system hallucinates that the size requirement is met. Moreover, Link2 and Link3 direct to non-Amazon websites, which violates the task constraint. In an attempt to explain the return policy, the system incorrectly extracts return information from these external sites. These issues reveal that when facing complex and fine-grained shopping queries, the OpenAI deep research system frequently exhibits hallucinations—both in satisfying attribute constraints and in sourcing policy information from inappropriate domains.

Task:
Locate the Return Policy for Women's Vintage Floral Maxi Dress on Amazon. Color: Navy Blue, Size: Medium. If free returns are available, please explain the return process for this item.

OPENAI deep research system answer:
Understood. I will search Amazon for listings of a Women's Vintage Floral Maxi Dress in Navy Blue, size Medium, and identify the return policy for each listing. I will focus on whether free returns are available and explain the return process for the item. I'll share the direct Amazon links for all relevant listings I find.

Research completed in 11m:

Return Policy for Women's Vintage Floral Maxi Dress on Amazon
Free Return Policies for Navy Blue Vintage Floral Maxi Dresses (Medium)
When shopping on Amazon for a women's vintage floral maxi dress in Navy Blue (size Medium), it's important to check if the item offers Free Returns. Below are a few popular listings that match this description, along with their return policy details:
Floerns Women's Sleeveless Halter Neck Vintage Floral Print Maxi Dress (Navy Blue)
Product: *Floerns Women's Sleeveless Halter Neck Vintage Floral Print Maxi Dress*
Color: "A Navy Flower",
Size: Medium.
Link: Amazon Product Page – Floerns Halter Neck Floral Maxi
Free Returns: Yes. This Floerns dress is Prime-eligible and marked with "FREE Returns", meaning you can return it at no cost if it doesn't fit or meet your expectations [Link1]. Amazon provides at least one prepaid return shipping option for eligible items, so returning this dress is hassle-free.
Return Process: You have 30 days from delivery to initiate a return via your Amazon account. Simply go to Your Orders, select this dress, and choose "Return or Replace Items."
Because it's eligible for free returns, Amazon will offer a prepaid shipping label or QR code for drop-off [Link2]. You can then package the dress (in new, unworn condition with tags) and drop it off at an approved location (e.g. UPS Store, Whole Foods, Kohl's, etc.) without paying return shipping [Link3]. Once the item is received, you'll get a full refund to your original payment method.

[Link1]: https://www.amazon.com/Floerns-Womens-Sleeveless-Halter-Vintage/dp/B0C3HQLBBQ#:~:text=,least%20one%20free%20return
[Link2]: https://www.smoking-er.com/store/rosemary-pellet#:~:text=,preferred%20free%20shipping%20option%20Drop%C2%A0off%C2%A0and%C2%A0leave
[Link3]: https://www.smoking-er.com/store/rosemary-pellet#:~:text=,preferred%20free%20shipping%20option%20Drop%C2%A0off%C2%A0and%C2%A0leave

Fig. 12. Illustration of hallucination errors in the OpenAI deep research system. The system returns three links for the query, including two that point to non-Amazon websites, and incorrectly assumes the returned products match the specified size and platform constraints.

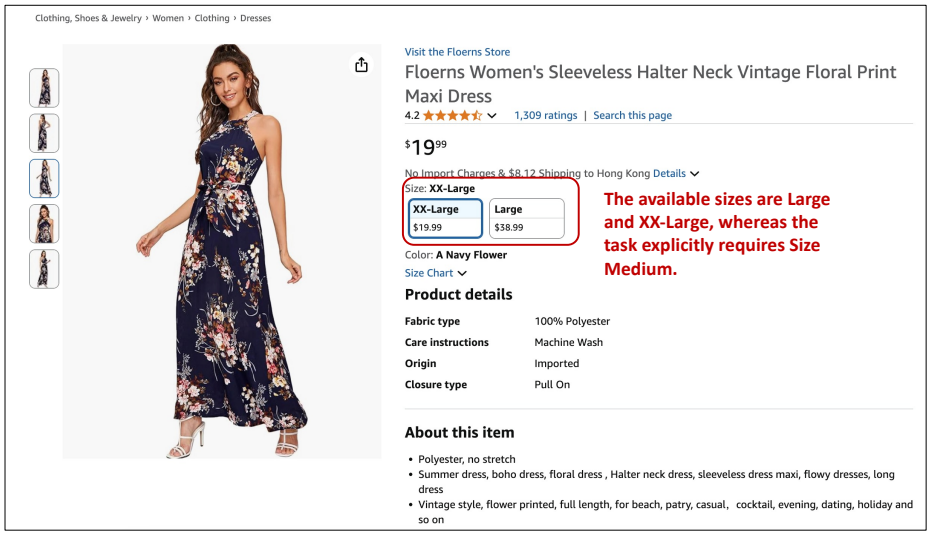


Fig. 13. Detailed view of the first returned product link. Although the task specifies size Medium, the linked product only offers Large and XX-Large options.

References

1. Abuelsaad, T., Akkil, D., Dey, P., Jagmohan, A., Vempaty, A., Kokku, R.: Agent-E: From autonomous web navigation to foundational design principles in agentic systems. arXiv preprint arXiv:2407.13032 (2024)
2. Gemini: Try deep research and Gemini 2.0 flash experimental. Gemini Blog (2025), https://blog.google/products/gemini/google-gemini-deep-research/
3. He, H., Yao, W., Ma, K., Yu, W., Dai, Y., Zhang, H., Lan, Z., Yu, D.: WebVoyager: Building an end-to-end web agent with large multimodal models. arXiv preprint arXiv:2401.13919 (2024)
4. Müller, M., Žunič, G.: Browser use = state of the art web agent (2024), https://browser-use.com/posts/sota-technical-report
5. OpenAI: Introducing deep research. OpenAI Blog (2025), https://openai.com/index/introducing-deep-research/
6. Xue, T., Qi, W., Shi, T., Song, C.H., Gou, B., Song, D., Sun, H., Su, Y.: An illusion of progress? Assessing the current state of web agents. arXiv preprint arXiv:2504.01382 (2025)
7. Zheng, B., Gou, B., Kil, J., Sun, H., Su, Y.: GPT-4V(ision) is a generalist web agent, if grounded. In: Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024. OpenReview.net (2024)